```
Output:
PHP Deprecated:  Function oldFunc() is deprecated since 1.0, Time to
migrate! ...
Use something else
```

The built-in attribute `#[Deprecated()]` is placed immediately before the declaration of `oldFunc()`, and when that function is called it emits an `E_USER_DEPRECATED` error. The attribute syntax looks suspiciously like an object instantiation, and that's exactly what it is: those named arguments are in fact constructor arguments. Behind the scenes, an instance of the `Deprecated` attribute class – which was given `message` and `since` constructor arguments – now lives in memory and PHP uses it whenever the attributed function is called. You can't make use of that `Deprecated` instance, it isn't "userland" code as such: think of attributes as a configuration meta-language that runs adjacent to the main program (because, well, that's exactly what they are). The full list of predefined attributes is as follows:

Table 6-4. *Built-in attributes*

`#[Attribute]`	Used to define your own attribute classes
`#[AllowDynamicProperties]`	Required in PHP >= 8.2 to mark classes allowing dynamic property (like `stdClass` does)
`#[Deprecated]`	Flag deprecated functionality
`#[Override]`	Mark a method as overriding the parent's, will emit error if no parent implementation
`#[ReturnTypeWillChange]`	Mark an overriding method to allow a different return type to parent
`#[SensetiveParameter]`	Mark method/function parameter data to be hidden in any stack trace

Prior to PHP 8 there were comment-based solutions (usually as *docblock* format comments `/** */`) which were not standardized or enforceable by the language and you'll see lots of things like this in older PHP…

```
/**
 * @SomeAnnotation
 */
```